JWT Invalidation

FastAPI Masterclass

JWT Invalidation I

- JWTs offer stateless authentication.
- A request does not know about any previous request.
- We can’t “delete” a JWT from a server because the server doesn’t save it.
- The JWT will expire after some time.

JWT Invalidation II

- Logout becomes the client's responsibility.
- When the user clicks a "Logout" button, the application (i.e., a React app) deletes the JWT/access token from wherever it is being stored.
- The client then has nothing to attach to the **Authorization** header, forcing the user to login again.

Drawbacks of JWTs

- The client handling the deletion of the token is the “happy path”. Your FastAPI backend and your frontend application code will work together for the user’s benefit.
- The challenge is reacting to a malicious actor getting a user’s JWT.
- If the backend needs to revoke a JWT before it expires, it must ultimately introduce some form of server-side state.

Server-Side State Example

- Store revoked JWTs in a database table/Redis.
- Add a JWT to the collection if we know it has been acquired by a hacker, thus marking it “revoked”.
- On every request, make sure JWT is not found in the “revoked” collection before allowing the request to proceed.